codegraph_mcp.sh — guide

Revision: 1 **Last modified:** 2026-09-26T12:55:57Z

Source: `constitution/scripts/codegraph/codegraph_mcp.sh` (140 lines). Tests: `constitution/scripts/codegraph/tests/test_codegraph_mcp.sh` (cases W1-W9) and its mutation harness `test_codegraph_mcp_mutations.sh`. Citations below are `codegraph_mcp.sh:<line>` unless another file is named.

Overview

The only sanctioned way to start a CodeGraph MCP server (header, lines 3-16). It starts a READ-ONLY server for one project so any agent or subagent can query the index without any chance of changing it. The reason it exists is that stock `codegraph serve --mcp` is not read-only: it runs a startup catch-up sync, runs schema/index heal DDL on open, steals a lock older than two minutes regardless of the holder, and spawns a detached daemon (lines 5-8; measured and described in `runner_patches/mcpro1.py:3-13`).

The wrapper therefore does three things (lines 9-16):

1. serves ONLY through the `mcpro1` runner, verified from its receipt; it never uses the `codegraph` found on `PATH` and never any other runner;
2. refuses to start while a live writer exists on the project's database, decided from real evidence (a process holding `codegraph.db` open for writing, or a live `codegraph`-looking process named in the lock file), never from the lock file's age;
3. pins the server environment (no daemon, no watcher, no update check, no telemetry, no prompt hook, query pool off, eight tools exposed) and then `exec`s the runner with `serve --mcp`, so the wrapper's stdout becomes the MCP JSON-RPC channel.

In this repository the wrapper is what `.mcp.json` registers for the `codegraph` server: `command: bash`, `args: ["constitution/scripts/codegraph/codegraph_mcp.sh"]` (`.mcp.json:6`). No `--project` is passed, so the target is `$CODEGRAPH_MCP_PROJECT` if set, otherwise the current working directory (lines 37, 48).

Prerequisites

- `bash`, `python3` (used for the runner-receipt check, the live-writer scan and, on first use, the runner builder), plus `sed`, `mktemp`, `head`, `tr` (lines 44, 54, 59, 63, 78).
- Linux `/proc` (the writer scan reads `/proc/<pid>/fd`, `/proc/<pid>/fdinfo`, `/proc/<pid>/cmdline`, lines 86, 90-110, 112-118). UNCONFIRMED: behaviour on a host without `/proc` was not run.
- An existing index: `<project>/.codegraph/codegraph.db` must be a regular file (lines 50-51). The wrapper never creates one.
- A way to obtain the `mcpro1` runner, either
  - an installed `@colbymchenry/codegraph` 1.6.x package to build it from (the builder finds it with `npm root -g`, `fk_index_patch.py:104-116`), or
  - a pre-built runner pinned through `HELIX_CODEGRAPH_MCP_RUNNER` (line 55).

Usage

```
codegraph_mcp.sh [--project DIR | --project=DIR] [--dry-run] [serve] [--mcp]
codegraph_mcp.sh -h | --help
```

Argument	Meaning	Source
`--project DIR`	Project root that holds `.codegraph/codegraph.db`. Default: `$CODEGRAPH_MCP_PROJECT`, then `$PWD`. A bare `--project` with no value exits 2	lines 37, 40, 48
`--pro-ject=DIR`	Same, single-word form. Accepted by the code, but not listed in the header's usage line	line 41
`--dry-run`	Do every check, print one JSON verdict line plus the `ENV` / `EXEC` plan, and do NOT start the server	lines 42, 132-137
`serve`, `--mcp`	Accepted and ignored, so an MCP client can declare `command=codegraph_mcp.sh args=[serve,--mcp]`	line 43
`-h`, `--help`	Print lines 2-32 of the script itself (the header block) and exit 0	line 44
anything else	Refused with exit 2 (`index`, `sync`, `init`, `uninit` are the usual cases)	line 45

Arguments are parsed left to right; `-h` and a bad argument take effect as soon as they are reached, before any project, runner or database is touched (lines 38-47).

Examples

#	Command	Status
1	`codegraph_mcp.sh --help`	VERIFIED (ran: `bash codegraph_mcp.sh --help`; saw: 31 header lines from `# ====` on, exit 0)
2	`codegraph_mcp.sh index`	VERIFIED (ran: `bash codegraph_mcp.sh index`; saw: `codegraph_mcp: refusing 'index': this entry point only serves MCP read-only ...`, exit 2)
3	`codegraph_mcp.sh --project` (no value)	VERIFIED (ran; saw: `--project needs a directory`, exit 2)
4	`codegraph_mcp.sh --project <dir without .codegraph> --dry-run`	VERIFIED (ran against an empty scratch directory; saw: `no CodeGraph index at <dir>/.codegraph/codegraph.db (the read-only server never creates one)`, exit 3; directory still empty afterwards)
5	`codegraph_mcp.sh --project <nonexistent dir>`	VERIFIED (ran; saw: `project directory not found`, exit 3)
6	`CODEGRAPH_MCP_PROJECT=<dir> codegraph_mcp.sh --dry-run` and `--project=<dir> serve --mcp --dry-run`	VERIFIED (ran against the same empty scratch directory; both reached the no-index refusal, exit 3, proving the env-var default and the `=` form are read)

#	Command	Status
7	`HELIX_CODEGRAPH_RUNNER_DIR=<scratch cache> HELIX_CODEGRAPH_SRC=<installed package> codegraph_mcp.sh --project <fixture> --dry-run`	VERIFIED (ran against a throwaway fixture project and a scratch runner cache built by the test suite; saw one JSON line `{"ok": true, "project": ..., "runner": ".../1.6.0-mcpro1/bin/codegraph", "runner_key": "1.6.0-mcpro1", "writers": []}`, then eight `ENV ...` lines, then `EXEC <runner> serve --mcp (cwd <project>)`, exit 0)
8	`HELIX_CODEGRAPH_MCP_RUNNER=/nonexistent/bin/codegraph codegraph_mcp.sh --project <fixture> --dry-run`	VERIFIED (ran on the same fixture; saw `runner not executable: /nonexistent/bin/codegraph`, exit 5)
9	`codegraph_mcp.sh --project <real project>` (starts a live server)	NOT RUN (a live server was deliberately never started against this repository's real `.codegraph`; the server path is covered by test cases W1 and W9, which start it on fixtures, see "Last verified")

Inputs

Environment variables read by the script:

Variable	Effect	Source
`CODEGRAPH_MCP_PROJECT`	Default project directory when `--project` is not given	line 37
`HELIX_CODEGRAPH_MCP_RUNNER`	Pin an explicit runner binary. Its receipt is STILL verified and must say exactly `mcpro1`	lines 24, 55-56, 63-75
`HELIX_CODEGRAPH_SRC`	Pristine package the runner is built from (passed as `--src` to the builder). Ignored when `HELIX_CODEGRAPH_MCP_RUNNER` is set	lines 23, 58
`HELIX_CODEGRAPH_RUNNER_DIR`	Runner cache directory (read by the builder, not by this script). Builder default `~/.cache/helix/code-graph_runner`	line 22; `fk_in-dex_patch.py:207-209`
`TMPDIR`	Where the wrapper's own temp file is created (default `/tmp`)	line 54

Nothing is read from stdin: once `exec` runs, stdin belongs to the MCP server.

Outputs

- stdout: only the MCP JSON-RPC stream produced by the server, or, with `--dry-run`, the plan below. The script itself never writes to stdout on a refusal (`die` writes to stderr, line 35).
- stderr: `codegraph_mcp: <message>` on any refusal (line 35).

`--dry-run` output (lines 133-135):

1. one JSON line: `{"ok": true, "project": "<dir>", "runner": "<bin>", "runner_key": "<key>", "writers": []}`
2. eight lines `ENV NAME=value`, one per pinned variable

3. one line `EXEC <runner> serve --mcp (cwd <project>)`

The pinned environment (lines 127-131), exactly:

```
CODEGRAPH_NO_DAEMON=1        CODEGRAPH_NO_WATCH=1
CODEGRAPH_NO_UPDATE_CHECK=1
DO_NOT_TRACK=1               CODEGRAPH_TELEMETRY=0
CODEGRAPH_NO_PROMPT_HOOK=1
CODEGRAPH_QUERY_POOL_SIZE=0
CODEGRAPH_MCP_TOOLS=explore,node,search,callers,callees,impact,files,statu
s
```

Without `--dry-run` the same variables are exported, the working directory is changed to the project, and the runner is exec'd: `exec "$RUNNER" serve --mcp` (lines 138-140).

Exit codes

Code	Meaning	Source
0	`--dry-run` completed; `--help` printed. A normal server start never returns: the process becomes the runner and the exit status is the runner's	lines 44, 136, 140
2	Usage: `--project` without a value, or any unsupported argument	lines 40, 45
3	The project directory does not exist, there is no `.code-graph/codegraph.db`, or `cd` into the project failed	lines 49, 51, 139
4	A live writer was found, OR the writer scan itself failed (refused conservatively)	lines 121-123
5	The runner cannot be provided, is not executable, its directory is unresolvable, or it is not a valid `mcpro1` runner	lines 59, 61, 62, 75

The header lists 0/2/3/4/5 (line 26); the exit 3 for "project directory not found" and "cannot enter" is part of the code but the header names only "no index".

Edge cases

- **Read-only means no index creation.** A project without `.codegraph/codegraph.db` is refused (exit 3) and nothing is created (lines 50-51; example 4 left the scratch directory empty).
- **`index`, `sync`, `init`, `uninit` are refused before anything else is resolved** (exit 2, lines 45; test W6).
- **No fallback to the `PATH` codegraph.** If the `mcpro1` runner cannot be built or found, the wrapper exits 5 instead of running whatever `codegraph` is on `PATH` (lines 55-60, error text "no fallback to the PATH codegraph"; test W5 plants a stub `codegraph` on `PATH` and asserts it never runs).
- **A pinned runner is not trusted on its name.** `HELIX_CODEGRAPH_MCP_RUNNER` still goes through the receipt check: `RUNNER_RECEIPT.json` in the runner's directory (the parent of `bin/`) must have `patch_set` exactly `["mcpro1"]`, a `runner_key` ending `-mcpro1`, the `helix-mcpro1` marker in five listed files, and a `db/index.js` whose SHA-256 equals `patched_dbjs_sha256` (lines 62-75). A writer runner is refused with exit 5 (test W8).
- **Writer detection uses the main database file only.** SQLite opens `-wal` and `-shm` read-write even for a read-only connection, so those descriptors are not treated as writer evidence (line 82, comment records the measurement). A process is a writer when it holds `codegraph.db` open with `flags & 3 != 0`, or when the flags are unreadable (treated as writable, lines 106-110).
- **Lock file: liveness is judged from `/proc`, never from age.** The first token of `.codegraph/codegraph.lock` is read as a PID; if that process is alive and its command line contains `codegraph` (case-insensitive) or is unreadable, it counts as a writer (`lock-holder-alive`). A live process that is not `codegraph` (a recycled PID) and a dead PID do not block (lines 111-118; test W4 covers an ancient lock plus live codegraph PID = refuse, live non-codegraph = allow, dead PID = allow).
- **The wrapper and its parent are excluded from the scan** (`os.getpid()`, `os.getppid()`, lines 83, 91, 115).
- **Other users' processes are invisible to the scan.** A process whose `/proc/<pid>/fd` cannot be listed is skipped silently (lines 94-97). This is a property of the code; UNCONFIRMED: not run with a second user.

- **Readers never block readers.** A live read-only MCP session holds no writable descriptor on the main file, so a second wrapper start is not refused (test W9).
- **Symlinks:** the project path is resolved with `pwd -P` and the database path with `realpath` before comparing with `/proc` link targets (lines 49, 81).
- **Refusal text is truncated:** the writer report is cut to 400 characters (line 123) and runner errors to 300 (lines 59, 75, 121).
- **Temp file left behind on a successful start (FINDING).** Line 54 creates an empty `${TMPDIR:-/tmp}/cg_mcp_err.XXXXXX` file and installs `trap 'rm -f "$ERRF"' EXIT`. On a successful start the script ends with `exec` (line 140), the shell is replaced, and the trap never runs, so one empty file is left behind per server start. Observed: after the test suite (two servers started, cases W1 and W9), exactly two empty `cg_mcp_err.*` files remained in the scratch `TMPDIR`, while every `--dry-run` and every refusal removed its own. The header's "Side effects" (lines 27-28) does not mention this file.

Internal behaviour

1. Parse arguments (lines 37-47); resolve the project to a physical path; require the database file (lines 48-51).
2. Create the temp error file and the EXIT trap (line 54).
3. Obtain the runner (lines 55-61): pinned path, or `python3 fk_index_patch.py --patches mcpro1 --print-bin [--src <HELIX_CODEGRAPH_SRC>]`. The builder reuses a runner whose receipt is valid or builds one by copying the installed package and applying the patch (`fk_index_patch.py:214-215, 225, 245`); it refuses to replace a runner directory that live processes are using (`fk_index_patch.py:218-220`). The runner is built only into the runner cache, never into the project.
4. Verify the runner receipt and marker files (lines 62-75); exit 5 on any assertion failure.
5. Scan `/proc` for writers of the main database file and check the lock holder (lines 78-124); exit 4 if any, or if the scan cannot finish.
6. `--dry-run`: print the plan and exit 0 (lines 132-137).
7. Export the pinned variables, `cd` into the project, `exec` the runner (lines 138-140).

What the `mcpro1` runner itself enforces (read-only open, no migrations or heals, no lock stealing, no daemon, CLI allowlist of `serve --mcp` and `status`) is described in `runner_patches/mcpro1.py:15-38`; this wrapper only guarantees that this runner, and no other, is what starts.

Related

File	Relation
`fk_index_patch.py`	Builds/reuses the `mcpro1` runner (`--patches mcpro1 --print-bin`)
`runner_patches/mcpro1.py`	The read-only patch the runner carries
`code-graph_mcp_preflight.sh`	Pre-flight check that uses this wrapper (see its guide)
codegraph_mcp_probe	MCP client used by the tests to prove read-only behaviour; reads descriptor modes from `/proc`
`codegraph_safe.sh`	The writer entry point: index/sync/init belong there, not here (line 45)
owed_report	Reports whether this tool has tests and a guide
`tests/test_code-graph_mcp.sh`	Cases W1-W9

Last verified

2026-09-26T12:55:57Z (source lines quoted above were read on that date).

- Ran `TMPDIR=<scratch> bash tests/test_codegraph_mcp.sh` (fixtures and a runner cache created under the scratch `TMPDIR`; the repository's real `.codegraph` was never opened): `W1`-`W9` all `PASS`, `SUMMARY cases=9 failed=0`.
- Ran the safe invocations in the examples table (help, usage errors, no-index and missing-directory refusals, dry-run on a scratch fixture, bad pinned runner).

- Counted leftover `cg_mcp_err.*` files in the scratch `TMPDIR` after the suite: 2 (the two servers started by W1 and W9); after an extra dry-run the count stayed 2 (control: the non-exec path leaves none).